Robot Movement Planning

A Constraint Programming approach to gripper-aware depalletizing

Alessio Russo

April 28, 2026

1 Problem statement

A continuous belt produces $x \times y$ boxes (with y orthogonal to the belt direction). A target packaging layer of n boxes is given by the destination positions $(P_x[i], P_y[i])$ for $i \in 1, \dots, n$, so that boxes do not overlap and tile a rectangle.

A robotic gripper of size S can pick a contiguous run of boxes along the x-axis whose total physical length is at most $S + x$ (i.e. up to $\lfloor (S + x)/x \rfloor$ boxes), with at most $x/2$ overflow per side. The gripper has two parallel plates: only *one* can open at any release, so the side that opens must face a free direction — either the boundary or a position that has not been filled yet. This generates two coupled subproblems:

(a) **Grouping.** Partition the n packs into contiguous, same-row groups, each fitting in the gripper.

(b) **Scheduling.** Order the groups in time and assign a plate side per group so that, when released, the opening plate faces an empty direction.

We encode (a) and (b) jointly in a single MiniZinc model, solved as a satisfiability problem.

2 Model

The MiniZinc model lives in `model.mzn`. We describe its parameters, decision variables, and constraints in detail.

2.1 Parameters and derived constants

```
par int: n_packs;
par int: packs_x_dim;
par int: packs_along_x;
par int: packs_along_y;
par int: grip_size;

par int: max_pkble_packs = (grip_size + packs_x_dim) div packs_x_dim;
par int: groups_per_row  = (packs_along_x + max_pkble_packs - 1)
                           div max_pkble_packs;
par int: n_groups        = packs_along_y * groups_per_row;
```

Listing 1: Parameters and derived constants.

Inputs. `n_packs` is the total number of boxes on the pallet. `packs_along_x` and `packs_along_y` are the grid dimensions of the placement layer. `packs_x_dim` is the box side parallel to the gripper (so `packs_x_dim` $= x$ in the natural orientation, or y if the layer is rotated). `grip_size` is S.

Derived.

- `max_pkble_packs` $= \lfloor (S + x)/x \rfloor$ is the gripper capacity in boxes, derived directly from the geometry.
- `groups_per_row` is the minimum number of groups needed to cover one row, computed by ceiling division.
- `n_groups` fixes the total number of groups. We use a *fixed* group cardinality to make the model fully static: every group is filled, and the partition is forced to be a contiguous prefix-sum sequence.

2.2 Decision variables

```
array[1..n_groups] of var 1..n_packs:          group_start;
array[1..n_groups] of var 1..max_pkble_packs: group_len;

array[1..n_groups] of var 1..n_groups: drop_order;
array[1..n_groups] of var 0..1:        plate;
```

Listing 2: Decision variables.

- `group_start[g]` – index of the first pack of group g in row-major order over the pallet grid.
- `group_len[g]` – number of packs in group g, bounded by the gripper capacity.
- `drop_order[g]` – the temporal step (1..n_groups) at which group g is dropped on the pallet.
- `plate[g]` – which plate opens when group g is released (0 = left plate, 1 = right plate).

2.3 Grouping constraints

```
constraint group_start[1] = 1;

constraint forall(i in 1..n_groups - 1)(
  group_start[i + 1] = group_start[i] + group_len[i]
);

constraint group_start[n_groups] + group_len[n_groups] - 1 = n_packs;

constraint forall(i in 1..n_groups)(
  (group_start[i] - 1)                div packs_along_x =
  (group_start[i] + group_len[i] - 2) div packs_along_x
);
```

Listing 3: Contiguity and row-boundary constraints.

The first three constraints together force the groups to form a contiguous partition of $\{1, \ldots, n\}$: the first group starts at pack 1, each subsequent group starts where the previous one ended, and the last group ends at pack n. This makes `group_start` a prefix-sum of `group_len`, which gives the solver a tight propagation handle.

The fourth constraint enforces the *same-row* requirement of the gripper: a group must not straddle a row boundary in the row-major indexing. Using integer division by `packs_along_x`, the row index of the first pack must equal the row index of the last pack.

2.4 Neighbor structure (parameter, not variable)

```
array[1..n_groups] of par int: left_neighbor =
  [ if (g - 1) mod groups_per_row > 0
    then g - 1 else 0 endif
  | g in 1..n_groups ];

array[1..n_groups] of par int: right_neighbor =
  [ if (g - 1) mod groups_per_row < groups_per_row - 1
    then g + 1 else 0 endif
  | g in 1..n_groups ];
```

Listing 4: Left/right neighbors as par arrays.

Because groups are filled in row-major order with a fixed `groups_per_row`, the spatial adjacency between groups is *known at compile time*: it is a function of the group index alone, not of `group_start` or `group_len`. We exploit this by declaring `left_neighbor` and `right_neighbor` as `par` (not `var`) arrays, computed from index arithmetic. A neighbor index of 0 is the sentinel for "no neighbor on that side" (i.e. row boundary). This removes a layer of decision variables and lets the flattener fully unroll the scheduling constraint below.

2.5 Scheduling constraints

```
constraint alldifferent(drop_order);

constraint forall(g in 1..n_groups)(
  (plate[g] = 0 /\
    (left_neighbor[g]  = 0 \/
     drop_order[left_neighbor[g]]  > drop_order[g]))
  \/
  (plate[g] = 1 /\
    (right_neighbor[g] = 0 \/
     drop_order[right_neighbor[g]] > drop_order[g]))
);

solve satisfy;
```

Listing 5: Plate-collision constraint and step permutation.

Permutation. `alldifferent(drop_order)` together with the domain $1..n\_groups$ forces `drop_order` to be a permutation: each step $1, \ldots, n\_groups$ is used exactly once.

Plate-collision. For each group g, we require that the *opening* plate faces a free side at the moment of release. "Free" means either:

- the relevant neighbor does not exist (sentinel 0, group is on a row boundary), or
- the neighbor exists but is dropped *later* than g, so the cell on that side is still empty.

Encoding this as a disjunction over `plate[g]` couples the two scheduling decisions: once `plate` is fixed for a group, the required temporal ordering with one specific neighbor is determined.

Solve. The model is a pure feasibility problem (`solve satisfy`); we do not optimize the schedule, since any ordering that respects the plate-collision constraint is a valid pick-and-place plan.

3 Driving the model from Python

The Python layer is intentionally thin — it is a wrapper around `minizinc-python` that handles the geometry computations the model deliberately leaves out, and produces visual artifacts of solutions. Three scripts compose the pipeline:

`sample.py` Solves a single instance and renders the layout and a step-by-step drop schedule as PDFs. The Python side is responsible for picking the orientation that fits more boxes on the pallet (`lib/utils.py`: `gen_pallet_info`), deriving `packs_x_dim` and the gripper size $S = 3x$, and translating the model output (the four arrays `group_start`, `group_len`, `drop_order`, `plate`) into a high-level *schedule* of drop actions.

`benchmark.py` Runs the same model across the cartesian product of five pallet presets and six pack sizes, against every available solver, with a 5-minute timeout. Each run is encapsulated and timing/error information is recorded in a single `benchmark/results.json` file.

`plot_benchmark.py` Consumes the JSON benchmark and produces the comparison plots referenced in Section 4.

The renderer in `lib/visualize.py` is orientation-agnostic: it consumes already-rotated cell dimensions (`cell_w`, `cell_h`) and the four solution arrays, and outputs both an overall pallet figure (colored by group) and one figure per drop step (with an arrow showing the opening plate side).

3.1 Worked example

To make the output of the model concrete, we run `sample.py` on a 70×70 pack and an 800×1200 pallet. The geometry helper chooses the rotation that maximises pack count, yielding an 11×17 grid ($n = 187$ packs). With $S = 3 \cdot 70 = 210$, the gripper capacity is $\lfloor (210 + 70)/70 \rfloor = 4$ packs, and each row of 11 packs is split into 3 groups of sizes $3 + 4 + 4$, for a total of $n\_groups = 51$.

Figure 2 samples four steps from the 51-step schedule produced by the model. The opening plate is highlighted by a red border on the active group and a blue arrow pointing in the direction the plate opens; the side it points to is always empty at the moment of release.

The schedule starts in the bottom-right corner and works back toward the top-left: at step 1 the gripper drops G_{51} opening to the left (its left neighbor G_{50} has not yet been placed), and the final step places G_1 opening to the left into the pallet edge (no left neighbor exists, so the boundary sentinel applies).

4 Benchmarking

4.1 Setup

The benchmark runs the MiniZinc model on every (pallet, pack) combination from the lists in `benchmark.py` (see Table 1), against every solver registered with the local MiniZinc installation, with a per-run timeout of 300 seconds.

The gripper size is fixed at $S = 3 \cdot$ `packs_x_dim`, the maximum allowed by the project specification.

4.2 Overall scaling

Figure 3 shows solve time as a function of the total number of packs on the pallet, on a logarithmic axis, for every solver across all instances. The geometric mean and min/max range per solver is summarised in Figure 4.

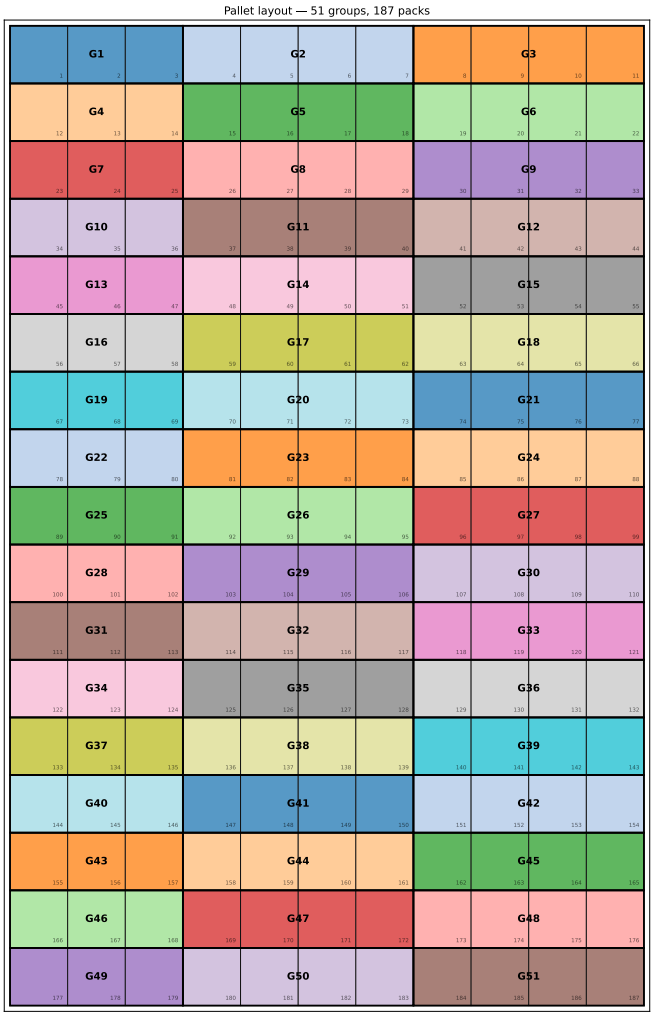


Figure 1: Pallet layout produced by the grouping stage: 51 groups tiling the 11×17 grid in row-major order, each colored distinctly. Groups alternate 3–4–4 across each row.

4.3 Per-pallet breakdown

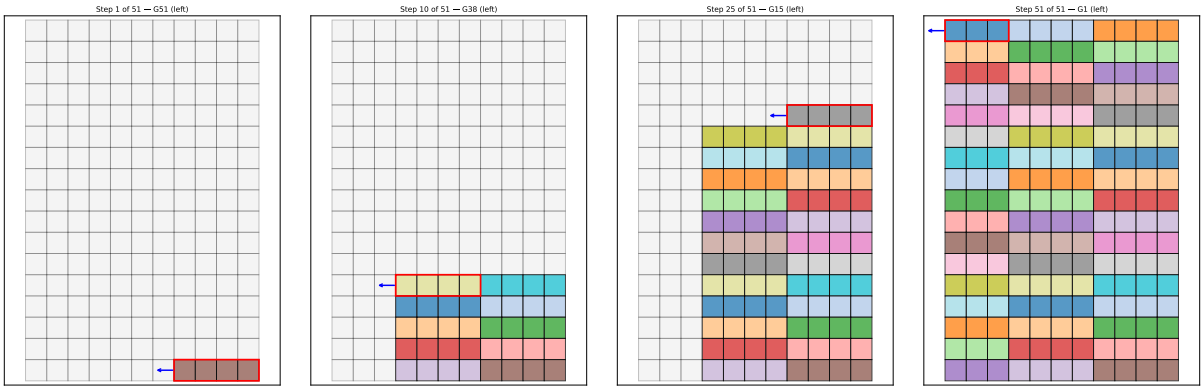
Splitting the data by pallet preset (Figure 5) isolates the effect of pack count from pallet shape: for a fixed pallet, the curves are functions of pack count alone, while differences between pallets reflect the impact of `packs_along_x` and `packs_along_y` on the number of groups.

4.4 Wall time vs. solve time

The gap between Python-measured wall time and solver-reported solve time captures MiniZinc flattening and inter-process overhead. Figure 6 plots the two against each other; points above the diagonal indicate runs in which the harness cost dominated the search itself.

5 Conclusion

The two-stage formulation — grouping the pallet into pickable runs, then scheduling the drops with a plate side per group — maps cleanly onto MiniZinc decision variables and yields a compact, fully feasible model. The key modelling choice is to fix the number of groups statically and force the partition into a prefix-sum shape; this lets us pre-compute the neighbor structure as parameters rather than variables, keeping the plate-collision constraint a simple disjunction over



(a) Step 1: G_{51}, bottom-right corner.

(b) Step 10: G_{38} opens left into a still-empty cell.

(c) Step 25: G_{15}, about halfway through.

(d) Step 51: G_1, the final drop, opens left into the pallet boundary.

Figure 2: Selected drop steps from the 51-step schedule. The red border marks the group being placed; the blue arrow shows the opening plate side, which always faces a free cell.

Pallet preset	**Dimensions (mm)**
Europallet	1200×800
Industrie	1200×1000
US 40×48	1219×1016
Half-pallet	800×600
Australian	1165×1165

Pack size (mm)
50×50
70×70
100×100
120×80
150×150
200×200

Table 1: Pallet presets and pack sizes used in the benchmark.

indexed lookups.

The benchmark confirms that the model scales well across realistic pallet sizes within the 5-minute budget, and exposes the expected per-solver trade-offs between flattening overhead and search performance.

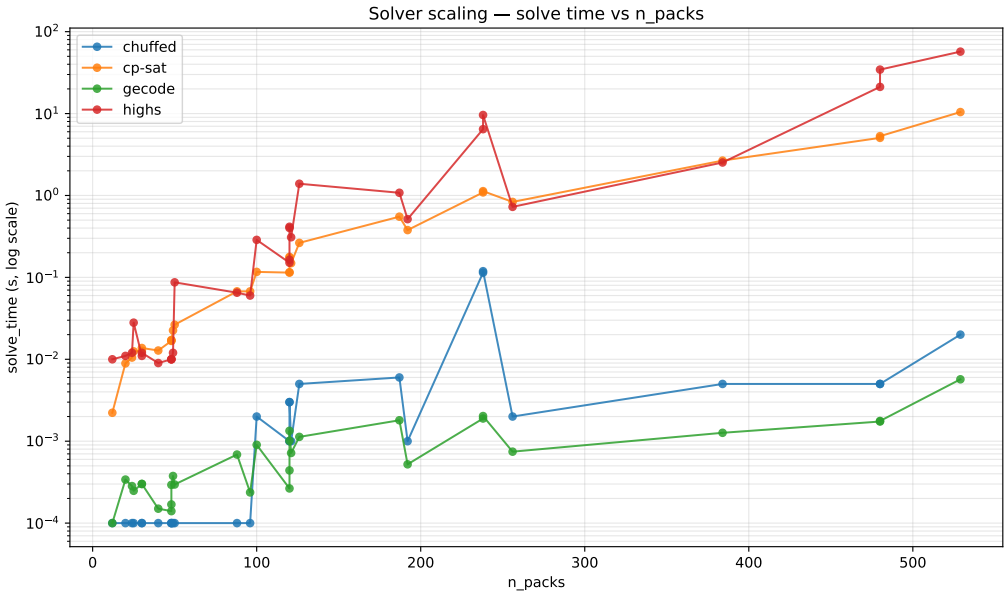


Figure 3: Solve time vs. total number of packs — all solvers, all instances. Log scale on the time axis.

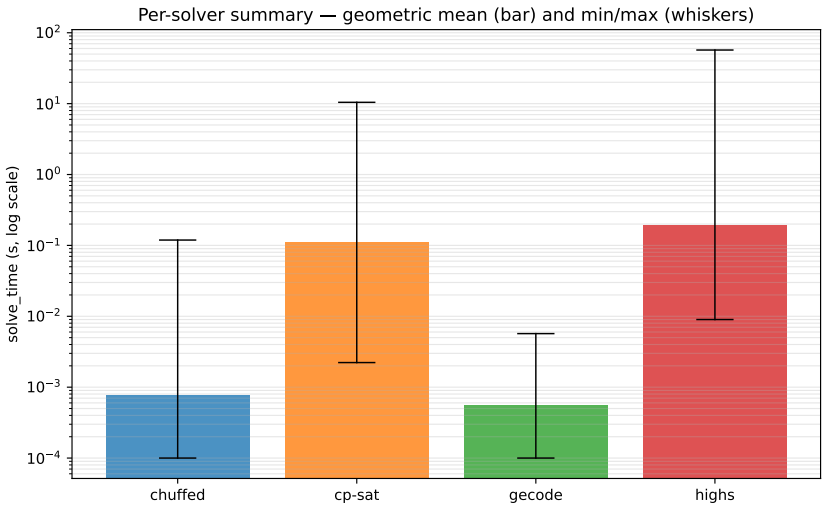
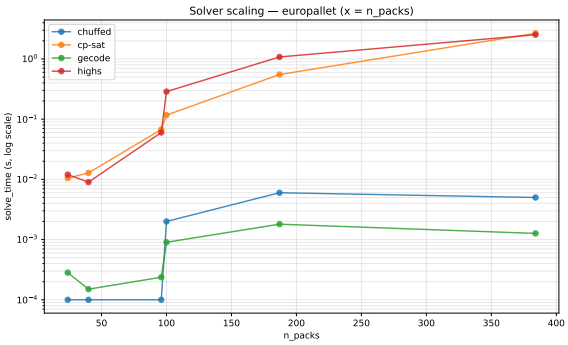
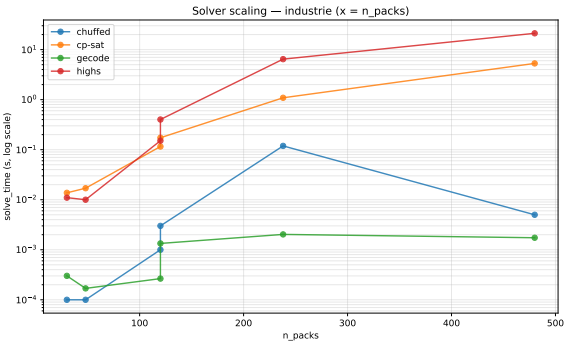


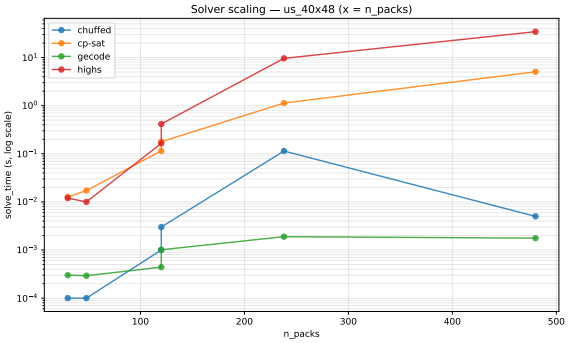
Figure 4: Per-solver geometric mean of solve time (bar) with min/max whiskers across the full benchmark set.



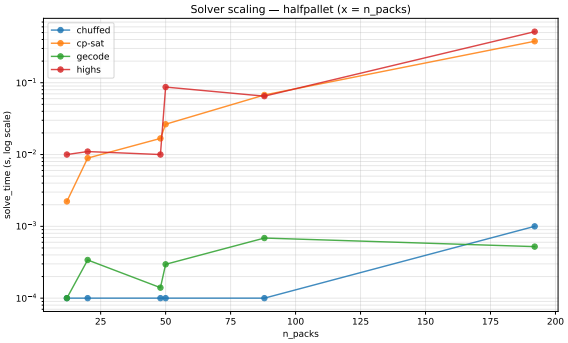
(a) Europallet



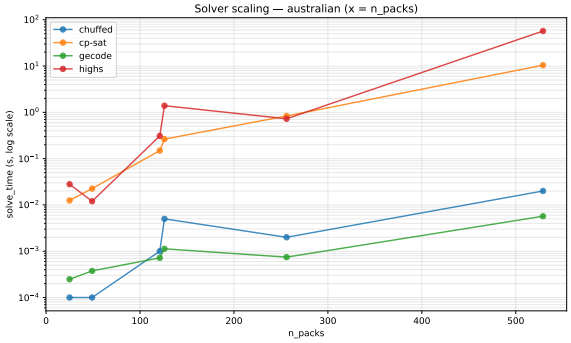
(b) Industrie



(c) US 40×48



(d) Half-pallet



(e) Australian

Figure 5: Solve time vs. number of packs, broken down by pallet preset.

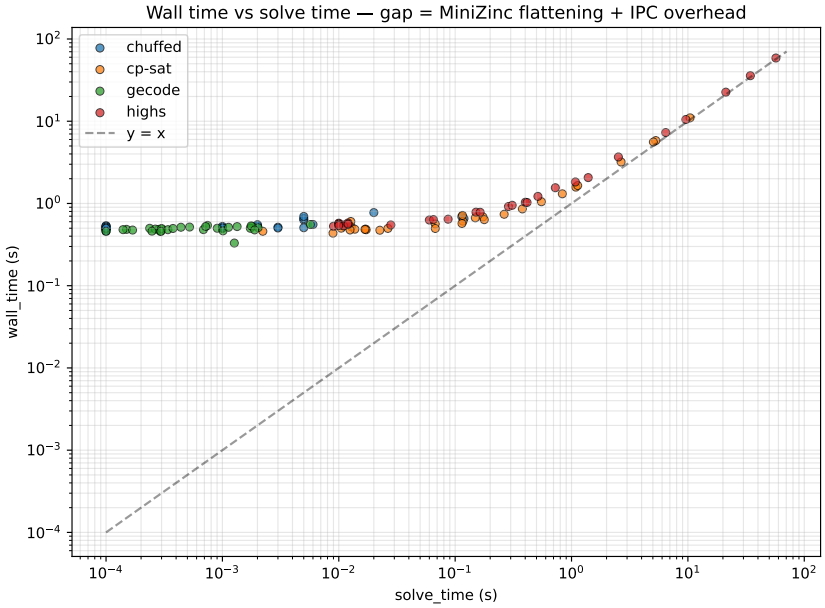


Figure 6: Wall time vs. solve time per run, log–log. The diagonal represents zero overhead; all points lie above it.